

LUSOHEALTH

RELATÓRIO TÉCNICO INDIVIDUAL

SISTEMA WEB PARA GESTÃO DE CRITICIDADE,
RASTREABILIDADE E DOCUMENTAÇÃO DE DISPOSITIVOS
MÉDICOS

DOCENTES:
Professor Pedro Guimarães
Professor Nuno Morgado

INÊS MARIA FERNANDES MOREIRA – 1241841 – TURMA 2^a
LEBIOM - Sistemas de Informação e Bases de Dados Aplicados à Saúde

JUNHO 2026



Resumo

O presente relatório descreve o desenvolvimento do projeto LusoHealth, realizado no âmbito da Unidade Curricular de Sistemas de Informação e Base de Dados Aplicados à Saúde (SIBDAS), do curso de Licenciatura em Engenharia Biomédica do Instituto Superior de Engenharia do Porto, no ano letivo 2025/2026.

O projeto teve como finalidade simular a atividade de uma empresa de desenvolvimento de software especializada em sistemas de informação para a área da saúde. Neste contexto, a sua implementação assentou em dois módulos distintos: *front office* e *back office*. Por um lado, o componente de *front office* consistiu na criação do website institucional da empresa de software. Aqui, foram incluídos conteúdos textuais e elementos visuais estáticos. Por outro lado, através do *back office* foi possível elaborar uma aplicação web funcional utilizada pelo hospital com o intuito de gerir o inventário dos equipamentos.

Deste modo, este projeto possibilitou a simulação de um contexto real de desenvolvimento de software aplicado à engenharia biomédica e aos sistemas de informação hospitalares.

Palavras-Chave

Front office, Back office, Frontend, Backend, Visual Studio Code, Laragon, GitHub, LusoHealth



Índice

Resumo.....	ii
Palavras-Chave.....	ii
Introdução	1
Contexto e Motivação	1
Objetivos do Projeto.....	1
Estrutura do Projeto	2
Organização do Código e Estrutura de Diretórios	2
Dois Contextos da Aplicação	4
Tecnologias Utilizadas	5
Tecnologias Utilizadas no Frontend	5
Tecnologias Utilizadas no Backend	6
Tecnologia Utilizadas na Base de Dados	6
Tecnologia de Controlo de Versões	6
Modelação da Base de Dados	7
Princípios de Modelação	7
Descrições das Tabelas.....	7
Normalização da Base de Dados (3ª Forma Normal – 3FN).....	8
Modelo Relacional, com restrições de integridade.....	9
Restrições de integridade não representáveis no modelo relacional Crow's Foot.....	10
Consultas SQL Aplicadas no Trabalho.....	13
Consulta com Junção de tabelas (JOIN)	13
Consulta com Subconsulta (subquery).....	13
Módulos da Aplicação.....	14
Módulo de Equipamentos	15
Módulo de Fornecedores	15
Módulo de Documentação	16
Módulo de Garantias e Módulo de Contratos	16
Dashboard e Gráficos	16
Conteúdos Públicos (Backoffice).....	16
Implementação de Mecanismos de Backend e Segurança.....	17
Autenticação e gestão de sessões.....	17
Validação e depuração de dados	17
Proteção contra SQL Injection.....	17
Encriptação e proteção de dados sensíveis.....	17
Gestão de sessões e controlo de acessos.....	18
Upload seguro de ficheiros	18



Documentação do Sistema de Scripts (JavaScript).....	19
Módulo 1: Equipamentos	19
Validação e formatação do código de inventário	19
Validação do ano de fabrico	19
Módulo 3: Comunicação assíncrona (AJAX / Fetch API).....	19
Módulo 4: Integração de bibliotecas jQuery (DataTables e notificações)	20
Módulo 5: Automação e geração de dados de teste	20
Módulo 6: Autenticação dinâmica (modo desenvolvimento)	20
Módulo 7: Componentes globais e geração de gráficos	20
Dificuldades Encontradas e Soluções Adotadas	21
Encriptação AES e Passagem de IDs nos URLs	21
AJAX para Criação de Categorias	21
Gestão de Sessões em Ficheiros Incluídos	21
Responsividade da Sidebar em Mobile	21
Exportação para PDF sem Bibliotecas Externas	21
Conclusão e Reflexão Crítica	22
Anexos	23
Anexo A – Logótipo Aplicação Web LusoHealth	23
Anexo B – Declaração de Uso de Inteligência Artificial	24

Índice de Figuras

Figura 1 - Estrutura de pastas mínima recomendada	2
Figura 2 - Estrutura de pastas utilizada no projeto	2
Figura 3 - Fluxo lógico da aplicação	4
Figura 4 - Gráfico do modelo relacional	9
Figura 5 – Restrição de Integridade não Representável no Modelo relacional Crow's Foot 1	11
Figura 6 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 2	11
Figura 7 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 3	11
Figura 8 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 4	11
Figura 9 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 5	12
Figura 10 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 6	12
Figura 11 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 7	12
Figura 12 - Consulta com junção de tabelas (JOIN).....	13
Figura 13 - Consulta com subconsulta (subquery)	14
Figura 14 - Logótipo LusoHealth	23



Índice de Tabelas

Tabela 1 - Descrição de tabelas da base de dados	7
--	---



Introdução

O presente relatório descreve o desenvolvimento do projeto LusoHealth, realizado no âmbito da Unidade Curricular de Sistemas de Informação e Base de Dados Aplicados à Saúde (SIBDAS), do curso de Licenciatura em Engenharia Biomédica do Instituto Superior de Engenharia do Porto, no ano letivo 2025/2026.

Contexto e Motivação

Atualmente, apesar da constante sensibilização para a organização e proteção de dados, em muitas instituições de saúde a gestão do inventário de equipamentos médicos, continua, ainda, a basear-se em folhas de *Excel* dispersas, documentos isolados, pastas físicas, variadas bases de dados sem ligação entre si, entre outros métodos. Neste contexto, o surgimento da implementação de sistemas de informação na saúde moderna reavaliou a dinâmica de tratamento de dados em contexto hospitalar, dado que possibilitou um controlo rigoroso, centralizado e digital do seu parque de equipamentos médicos. Para além disso, existem múltiplos riscos associados a uma má gestão de equipamentos, nomeadamente, falhas em dispositivos médicos em ambiente cirúrgico, perda de garantias e documentação técnica em falta. Desta forma a implementação em centros hospitalares, de sistemas de informação na saúde tornou-se imprescindível.

Neste enquadramento, surgiu uma unidade hospitalar de média dimensão que geria o seu parque tecnológico, com cerca de 1500 equipamentos médicos. Porém, a gestão era feita através de folhas Excel dispersas, registos manuais em papel, listas separadas de fornecedores e documentação técnica sem estrutura centralizada ou integridade referencial. Assim, inspirada neste desafio surgiu o projeto LusoHealth, uma aplicação web centralizada, acessível via navegador, onde é possível organizar, consultar e atualizar a informação relativa aos seus equipamentos médicos. No Anexo A – Logótipo Aplicação Web LusoHealth está presente o logótipo da aplicação web gerado com utilizando uma ferramenta de inteligência artificial.

Objetivos do Projeto

A aplicação foi desenvolvida com os seguintes objetivos funcionais:

- Registrar e gerir equipamentos médicos com informação completa de inventário, localização e criticidade clínica.
- Associar fornecedores, fabricantes e entidades de assistência técnica a cada equipamento.
- Gerir documentação técnica (manuais, certificados, relatórios) e contratos de garantia/manutenção.
- Disponibilizar uma dashboard com indicadores estatísticos e gráficos em tempo real.
- Gerir os conteúdos do site público institucional sem necessidade de editar o HTML.
- Garantir a segurança dos dados com autenticação, gestão de sessões e validação dupla de inputs.



Estrutura do Projeto

Nesta secção é apresentada a organização estrutural da aplicação desenvolvida, com especial enfoque na arquitetura de diretórios, separação funcional dos seus componentes e divisão entre os contextos público e privado do sistema. Será analisada a forma como os diferentes módulos foram organizados para garantir modularidade, segurança, escalabilidade e facilidade de manutenção, evidenciando igualmente a conformidade da solução implementada com as boas práticas de desenvolvimento web e com os requisitos estruturais definidos para o projeto.

Organização do Código e Estrutura de Diretórios

A arquitetura de ficheiros do projeto lusohealth garante uma separação clara entre recursos estáticos (*assets*), ficheiros de configuração do sistema, scripts de base de dados e conteúdos públicos e privados. Esta estrutura, para além de facilitar a manutenção do código, melhorar a segurança (isolando o núcleo da aplicação da raiz pública) e obedecer às boas práticas de desenvolvimento web, também se encontra em conformidade com a estrutura obrigatório fornecida pelos professores, representada na Figura 1 - Estrutura de pastas mínima recomendada.

```
/nomedoprojeto/  
├─ public/  
├─ private/  
├─ assets/  
│   ├─ css/  
│   ├─ js/  
│   └─ img/  
│   └─ ...  
└─ README.md
```

Figura 1 - Estrutura de pastas mínima recomendada

A árvore de diretórios do projeto assume a estrutura organizada representada na seguinte figura:

```
/lusohealth/  
├─ assets/  
│   ├─ bootstrap/  
│   ├─ css/  
│   ├─ datatables/  
│   ├─ flatpickr/  
│   ├─ fontawesome/  
│   └─ webfonts/  
├─ img/  
├─ includes/  
│   └─ sidebar/  
├─ jquery/  
├─ js/  
├─ config/  
├─ database/  
├─ private/  
│   ├─ includes/  
│   └─ views/  
│       ├─ conteudos/  
│       ├─ documentacao/  
│       ├─ equipamentos/  
│       ├─ fornecedores/  
│       ├─ garantias/  
│       └─ localizacoes/  
├─ public/  
├─ uploads/  
├─ commits.txt  
├─ commits_formatado.txt  
├─ README.txt  
└─ index.php
```

Figura 2 - Estrutura de pastas utilizada no projeto



Neste contexto, o projeto encontra-se inteiramente alocado dentro da diretoria principal `/lusohealth/`, a qual funciona como o contentor central da aplicação. Esta raiz é constituída pelas pastas estruturais `assets/`, `config/`, `database/`, `private/`, `public/` e `uploads/`, bem como por um conjunto de ficheiros individuais com funções específicas de suporte e controlo. Entre estes, destacam-se o `index.php`, que serve de ponto de entrada unificado e reencaminhamento principal do portal; o `README.txt`, responsável pela documentação técnica e identificação do autor; e os ficheiros `commits.txt` e `commits_formatado.txt`, que asseguram o registo estruturado e o controlo de versões do histórico de desenvolvimento em Git.

Na pasta `assets/`, encontram-se centralizados todos os recursos estáticos e bibliotecas de terceiros necessários para o funcionamento e estilização da interface do utilizador. Esta diretoria subdivide-se em componentes como `bootstrap/`, responsável pelo design responsivo, `css/` e `js/`, que alojam os estilos e scripts personalizados e pacotes especializados como o `datatables/` e o `flatpickr/`, utilizados para otimizar as listagens e a gestão de datas do inventário. Deste modo, todos os recursos externos (Bootstrap, Font Awesome, DataTables, Flatpickr) foram integrados localmente, eliminando dependências de CDNs externos e garantindo o funcionamento *offline* da aplicação no servidor do ISEP.

Na pasta `config/`, encontram-se centralizado o ficheiro responsável pela parametrização global do sistema e, fundamentalmente, pelo estabelecimento da ligação segura à base de dados. É nesta diretoria que reside a lógica de instanciação do objeto PDO, garantindo que as credenciais de acesso não estão dispersas pelo código e que a conectividade com o servidor MySQL é gerida de forma centralizada e eficiente através de um único ficheiro de configuração incorporado quando necessário.

Na pasta `database/`, estão armazenados os scripts SQL (`.sql`) desenvolvidos para a modelação da base de dados relacional. Esta pasta inclui tanto o script de criação da estrutura de tabelas (definição de entidades, chaves primárias, chaves estrangeiras e restrições de integridade) como os scripts de povoamento inicial, que introduzem dados realistas no sistema (como equipamentos médicos padrão, serviços hospitalares e utilizador de teste) para viabilizar a validação das funcionalidades da plataforma. Para além disso encontra-se um ficheiro PDF e a representação gráfica do modelo relacional.

Na pasta `private/`, está implementado o núcleo centralizado e protegido do *backoffice*, cujo acesso é estritamente vedado a utilizadores não autenticados. Esta diretoria subdivide-se em duas pastas essenciais. A pasta `includes/` armazena os componentes de código PHP reutilizáveis no ecossistema privado (como processamentos lógicos e verificações de segurança) e a `views/` organiza as interfaces visuais e os formulários operacionais das componentes do inventário. Para garantir a modularidade e uma arquitetura limpa, a pasta `views/` segmenta-se em submódulos especializados:

- `conteudos/`, para a moderação e atualização dos dados exibidos na área pública;
- `documentacao/`, para associar manuais técnicos e fichas de segurança aos dispositivos;
- `equipamentos/`, que processa o ciclo de vida e o CRUD (*Create, Read, Update, Delete*) dos dispositivos médicos;
- `fornecedores/`, responsável pela gestão das entidades externas e contactos;
- `garantias/`, que monitoriza os prazos de cobertura legal e contratual dos equipamentos;
- `localizacoes/`, que mapeia os aparelhos pelas respetivas salas, blocos ou serviços clínicos do hospital.



Na pasta `public/`, encontram-se alojados todos os ficheiros de livre acesso na Web, servindo de interface inicial para qualquer visitante. Esta diretoria engloba a página principal de apresentação da instituição de saúde e o formulário de autenticação (*login*). É a única área estrutural concebida para interagir diretamente com utilizadores anónimos, encaminhando-os para a área restrita após a validação bem-sucedida das suas credenciais.

Na pasta `uploads/`, encontra-se o repositório físico destinado a armazenar de forma organizada todos os ficheiros e documentos submetidos pelos utilizadores através da aplicação. Esta pasta isola os ficheiros dinâmicos (como PDFs de manuais técnicos ou imagens dos equipamentos) dos scripts de execução do sistema, garantindo uma política de organização rigorosa e prevenindo a sobreposição de ficheiros de código fonte.

Dois Contextos da Aplicação

A aplicação web desenvolvida baseou-se numa arquitetura cliente-servidor tradicional na Web, dividindo-se estritamente nas duas grandes áreas fundamentais pública e privada.

Por um lado, a área pública corresponde ao *frontend* de apresentação. Esta componente constitui o ponto de entrada da aplicação, sendo acessível por qualquer utilizador sem necessidade de autenticação prévia. Esta foi idealizada para integrar conteúdos realistas relacionados com a instituição, serviços clínicos e contactos. Para além disso, toda a informação exibida nesta área é carregada dinamicamente a partir do servidor, evitando, deste modo, a edição estática e direta de ficheiros HTML.

Por outro lado, a área privada representa o *backoffice* e torna possível a gestão do inventário. Esta destaca-se pela sua restritividade, sendo protegida por mecanismos de controlo de acesso. O acesso é exclusivo ao utilizador autenticado que possua credenciais válidas guardadas no sistema. No interior desta componente encontra-se a Dashboard principal de gestão do inventário clínico, que agrega métricas e estatísticas cruciais para a engenharia biomédica hospitalar. Para além disso, processam-se as operações CRUD (*Create, Read, Update, Delete*) associadas ao ciclo de vida dos equipamentos médicos e dos seus fornecedores, bem como a parametrização e moderação dos conteúdos da própria área pública.

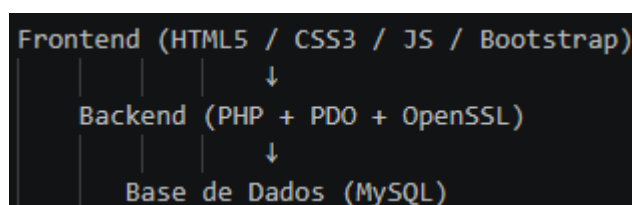


Figura 3 - Fluxo lógico da aplicação

Este modelo ilustra a separação clara de responsabilidades entre camadas, permitindo a manutenção simplificada do sistema. Ainda, a camada de *frontend* dinâmico complementa esta arquitetura através de scripts JavaScript que asseguram interatividade, validação em tempo real e comunicação assíncrona com o *backend*.



Tecnologias Utilizadas

As tecnologias escolhidas para a construção da aplicação foram as abordadas em aula. Assim, não só se cumpriu os requisitos obrigatórios da unidade curricular, como também se assegurou o correto funcionamento de todas as camadas da aplicação, desde a interface até à persistência de dados.

Tecnologias Utilizadas no Frontend

Na etapa da realização do *frontend* foram utilizadas variadas tecnologias, nomeadamente, HTML5, CSS3, Bootstrap 5, JavaScript, Chart.js, DataTables.js, Flatpickr e Font Awesome 6. Deste modo, a opção de integrar todas as bibliotecas localmente (em vez de CDNs) foi uma decisão deliberada para garantir o funcionamento estável no ambiente de servidor da ISEP, onde o acesso a recursos externos pode estar limitado.

Em primeiro lugar, o HTML5 auxiliou na construção da estruturação semântica global de todas as páginas do sistema, garantindo que a informação textual, tabelas de dados e elementos de formulário são interpretados de forma clara e padronizada pelos navegadores web.

Em segundo lugar, o CSS3 foi aplicado no desenho estilístico, tipografia e identidade visual da plataforma. Conforme regulamentado, a personalização estética assenta num ficheiro de estilos próprio (1241841.css), assegurando uma apresentação visual homogênea, profissional e cuidada.

Em terceiro lugar, Bootstrap representa um *framework* CSS integrado para acelerar o desenvolvimento de interfaces modernas e, fundamentalmente, garantir a total responsividade (*Responsive Web Design*) do portal. Desta forma, o inventário hospitalar adapta-se automaticamente a diferentes resoluções de ecrã (computadores de secretária, *tablets* e dispositivos móveis).

Em quarto lugar, o JavaScript foi implementado no lado do cliente com o objetivo de introduzir dinamismo e interatividade à interface, reduzindo a dependência de processamento constante no servidor. Foi utilizado na validação prévia de formulários, na manipulação dinâmica do DOM, na inicialização de bibliotecas como DataTables e Flatpickr, bem como na implementação de funcionalidades assíncronas, nomeadamente através de pedidos AJAX para carregamento dinâmico de categorias e de botões de preenchimento automático, recorrendo ao script autónomo *1241841.js*.

Em quinto lugar, o Chart.js foi utilizado para a representação visual de dados estatísticos na dashboard, permitindo a geração de gráficos informativos, nomeadamente gráficos de barras por serviço, gráficos circulares (*doughnut charts*) por edifício e gráficos de barras relativos à distribuição de equipamentos de suporte de vida.

Em sexto lugar, o DataTables foi integrado para enriquecer a experiência de navegação em tabelas, disponibilizando funcionalidades avançadas como pesquisa instantânea, ordenação dinâmica e paginação processada no lado do cliente.

Em sétimo lugar, o Flatpickr foi implementado como seletor de datas nos formulários de equipamentos, documentação e garantias, proporcionando uma introdução de datas mais intuitiva, consistente e menos suscetível a erros de inserção manual.

Em último lugar, o Font Awesome foi utilizado para incorporar ícones em toda a interface gráfica, incluindo barras de navegação, botões de ação e cartões da dashboard, contribuindo para uma experiência visual mais clara, moderna e intuitiva.



Tecnologias Utilizadas no Backend

Na etapa de desenvolvimento do *backend* foram utilizadas diversas tecnologias fundamentais para assegurar a lógica de negócio, a segurança e a comunicação com a base de dados, destacando-se o PHP, o PDO (*PHP Data Objects*) e o OpenSSL.

Em primeiro lugar, o PHP corresponde à principal linguagem de programação executada no lado do servidor, funcionando como o núcleo lógico da aplicação. Esta tecnologia foi responsável por processar as requisições provenientes do cliente, gerir o fluxo de navegação entre páginas, validar dados submetidos no servidor, controlar sessões de utilizador autenticado e executar operações críticas, como o processamento de formulários e o upload seguro de ficheiros.

Em segundo lugar, para estabelecer a comunicação entre o PHP e a base de dados, recorreu-se ao PDO. Esta interface de abstração orientada a objetos permitiu uniformizar o acesso aos dados de forma robusta e segura. A sua utilização revelou-se essencial para a implementação sistemática de *prepared statements* com associação de parâmetros, mecanismo que impede a injeção direta de dados nas instruções SQL e reforça significativamente a proteção da aplicação contra vulnerabilidades de *SQL Injection*.

Em último lugar, foi utilizada a biblioteca OpenSSL para reforçar a segurança da aplicação através da cifragem de identificadores transmitidos nos URLs. Este mecanismo reduz o risco de enumeração sequencial de registos e dificulta tentativas de acesso indevido a recursos internos do sistema.

Tecnologia Utilizadas na Base de Dados

Na etapa de desenvolvimento da base de dados foi utilizada a tecnologia MySQL, que constitui o Sistema de Gestão de Bases de Dados Relacionais (SGBDR) responsável pelo armazenamento persistente, estruturado e eficiente de toda a informação associada ao ecossistema hospitalar, incluindo tabelas de equipamentos, fornecedores, utilizadores e localizações. A base de dados encontra-se alojada no servidor *vsgate-s1.dei.isep.ipp.pt*, garantindo acesso remoto seguro e centralizado aos dados da aplicação. O MySQL assegura ainda o cumprimento rigoroso dos princípios de integridade referencial através da utilização consistente de chaves primárias e chaves estrangeiras, complementadas por mecanismos de validação como tipos ENUM, restrições CHECK e políticas de integridade referencial com ações *CASCADE* e *SET NULL*. Esta arquitetura permite suportar de forma robusta, normalizada e segura todas as operações de consulta e manipulação de dados mediadas pela camada PDO.

Tecnologia de Controlo de Versões

Por fim, foi utilizado o sistema de controlo de versões Git para assegurar o acompanhamento contínuo da evolução do projeto ao longo do semestre. A sua utilização permitiu manter um histórico progressivo e estruturado do desenvolvimento, materializado em mais de 30 *commits*, possibilitando o registo detalhado de alterações, a rastreabilidade das diferentes fases de implementação e um controlo mais rigoroso sobre a evolução do código-fonte.



Modelação da Base de Dados

Nesta secção é descrito o processo que culminou na criação da base de dados, bem como o seu respetivo modelo relacional, com as suas restrições de integridade. Em adicional, são apresentadas as restrições de integridade que não podem ser representadas no modelo relacional. Por fim, são apresentadas e explicadas duas consultas SQL implementadas no trabalho, a primeira corresponde a uma junção de tabelas e a segunda representa uma subconsulta.

Princípios de Modelação

A base de dados foi desenhada em conformidade com a Terceira Forma Normal (3NF), eliminando dependências transitivas e garantindo a integridade referencial. O modelo relacional foi representado com notação Crow's Foot e posteriormente transcrito para DBML, a partir do qual foi gerado o script DDL.

A base de dados encontra-se hospedada no servidor da ISEP (vsgate-s1.dei.isep.ipp.pt:10464), na base de dados db1241841.

Descrições das Tabelas

Na TABELA 1 encontra-se as tabelas implementadas na base de dados, bem como as suas respetivas colunas.

Tabela 1 - Descrição de tabelas da base de dados

Tabela	Colunas Principais	Notas
equipamentos	id, codigo_inventario, designacao, categoria_id, marca, modelo, num_serie, ano_fabrico, data_aquisicao, custo_aquisicao, tipo_entrada, estado, criticidade, localizacao_id, observacoes, criado_em, atualizado_em	Entidade central. ENUMs para estado (6 valores) e criticidade (4 valores). CHECK(custo_aquisicao >= 0).
fornecedores	id, nome, nif, tipo, telefone, email, morada, website, tecnico_nome, tecnico_telefone, observacoes, arquivado, criado_em, atualizado_em	NIF UNIQUE. ENUM tipo: Fabricante, Distribuidor, Assistência Técnica, Consumíveis. Flag arquivado = soft delete.
equipamento_fornecedor	equipamento_id, fornecedor_id, tipo_relacao	Tabela pivot N:N. PK composta. ENUM tipo_relacao: Fabricante, Distribuidor, Assistência técnica.
localizacoes	id, codigo, nome, edificio, piso, responsavel, observacoes, arquivado, criado_em, atualizado_em	Código UNIQUE (ex: HOS-UCIP). Piso TINYINT. Flag arquivado.
documentos	id, equipamento_id, fornecedor_id, tipo, nome_documento, data_documento, data_validade, notas, ficheiro_path, url_externo, arquivado, criado_em	FK equipamento NOT NULL. FK fornecedor NULL. Aceita path local ou URL externo.
garantias	id, num_contrato, equipamento_id, fornecedor_id, tipo, data_inicio, data_fim, periodicidade, ficheiro_path, url_externo, observacoes, arquivado, criado_em, atualizado_em	Num_contrato UNIQUE. CHECK(data_fim > data_inicio). ENUM tipo: Garantia de Fábrica, Contrato de Manutenção, Outro.



categorias	id, nome	Nome UNIQUE. Criação dinâmica via AJAX no formulário de equipamentos.
conteudos_publicos	id, chave, valor, atualizado_em	Chave UNIQUE. Permite edição dinâmica dos textos do site público sem alterar HTML.
utilizadores	id, username, password, criado_em	Password armazenada com password_hash() (bcrypt). Username UNIQUE.

Normalização da Base de Dados (3ª Forma Normal – 3FN)

A terceira forma normal (3FN) surgiu na necessidade de criar um método de normalização de bases de dados onde se evitasse o caos de uma base de dados repleta de informações repetidas e redundantes. Ao implementar-se a terceira forma normal, obtém-se uma estrutura limpa de dados, o que garante a eficiência, organização e remoção de dependências indesejadas da mesma. Neste sentido, a terceira forma normal baseia-se na primeira forma normal (1FN) e na segunda forma normal (2FN).

Assim sendo, a primeira forma normal exige valores individuais em cada célula, ou seja, não permite valores repetidos numa única coluna. Já na segunda forma normal não podem existir dependências parciais, isto é, em tabelas com chave composta, todo o atributo não-chave depende da chave inteira, não apenas de parte dela. Por fim, a terceira forma exige que não existam dependências transitivas, por outras palavras, um atributo não-chave não pode depender de outro atributo não-chave; só pode depender diretamente da chave primária.

Neste contexto, para a implementação da base de dados deste projeto foram aplicadas as regras descritas anteriormente. Assim, respeitando a primeira regra normal eliminou-se grupos repetidos, respeitando a segunda regra normal eliminou-se dependências parciais e respeitando a terceira regra normal eliminou-se dependências transitivas e redundantes de dados. As diversas aplicações das formas normais estão representadas nos seguintes tópicos:

- **Primeira forma normal:** Um equipamento pode estar associado a **vários** fornecedores (fabricante, distribuidor, assistência técnica) e a **vários** documentos. Assim, criar colunas repetidas como fornecedor_1, fornecedor_2, fornecedor_3 na tabela equipamentos violaria a primeira forma normal. Logo, para contornar este problema foi criada a tabela de associação equipamento_fornecedor, e as tabelas independentes documentos e garantias, cada uma ligada a equipamentos por chave estrangeira. Isto permite um número ilimitado de fornecedores e documentos por equipamento, sem alterar a estrutura da tabela.
- **Segunda forma normal:** A tabela equipamento_fornecedor foi garantida a 2FN dado que não existem dependências parciais nesta tabela. Na tabela referida, existe uma chave primária composta (equipamento_id, fornecedor_id). Mas o único atributo não-chave, tipo_relacao, depende da combinação completa das duas colunas, ou seja, o papel do fornecedor X só faz sentido associado ao equipamento Y específico, não dependendo, deste modo, apenas de equipamento_id nem apenas de fornecedor_id isoladamente.
- **Terceira forma normal:** Este foi o critério mais relevante na modelação do projeto. Em vez de armazenar repetidamente os dados de cada fornecedor (nome, NIF, telefone, email) em todas as linhas de equipamentos, documentos e garantias onde esse fornecedor é referido, criou-se uma única tabela fornecedores, e as restantes tabelas referenciam-na através da chave estrangeira fornecedor_id. O mesmo princípio foi aplicado às categorias onde ao invés de se escrever o nome da categoria (ex: "Equipamentos de Monitorização") diretamente em cada linha de equipamentos, existe a tabela categorias e a coluna categoria_id faz a referência.



Este princípio também foi aplicado às localizações onde o nome do serviço, edifício e piso não são repetidos em cada equipamento; estão centralizados na tabela localizacoes, referenciada por localizacao_id.

Concluindo, esta normalização apresenta inúmeras vantagens diretas para a implementação da base de dados. Numa primeira instância, permite que não haja redundância, tome-se como exemplo: o NIF e o telefone da Philips Medical Systems Portugal estão guardados uma única vez, mesmo que dezenas de equipamentos tenham esse fornecedor associado. Numa segunda estância, o código não apresenta inconsistências, ou seja, se o telefone de um fornecedor mudar, basta atualizar na tabela fornecedores e todos os equipamentos associados refletem automaticamente o novo valor, sem risco de ficarem dessincronizados. Numa terceira instância, a integridade da base de dados é garantida, onde as chaves estrangeiras impedem que um equipamento referencie uma categoria, localização ou fornecedor inexistente.

Modelo Relacional, com restrições de integridade

A seguinte imagem foi gerada através do site <https://dbdiagram.io/home> e representa o gráfico do modelo relacional.

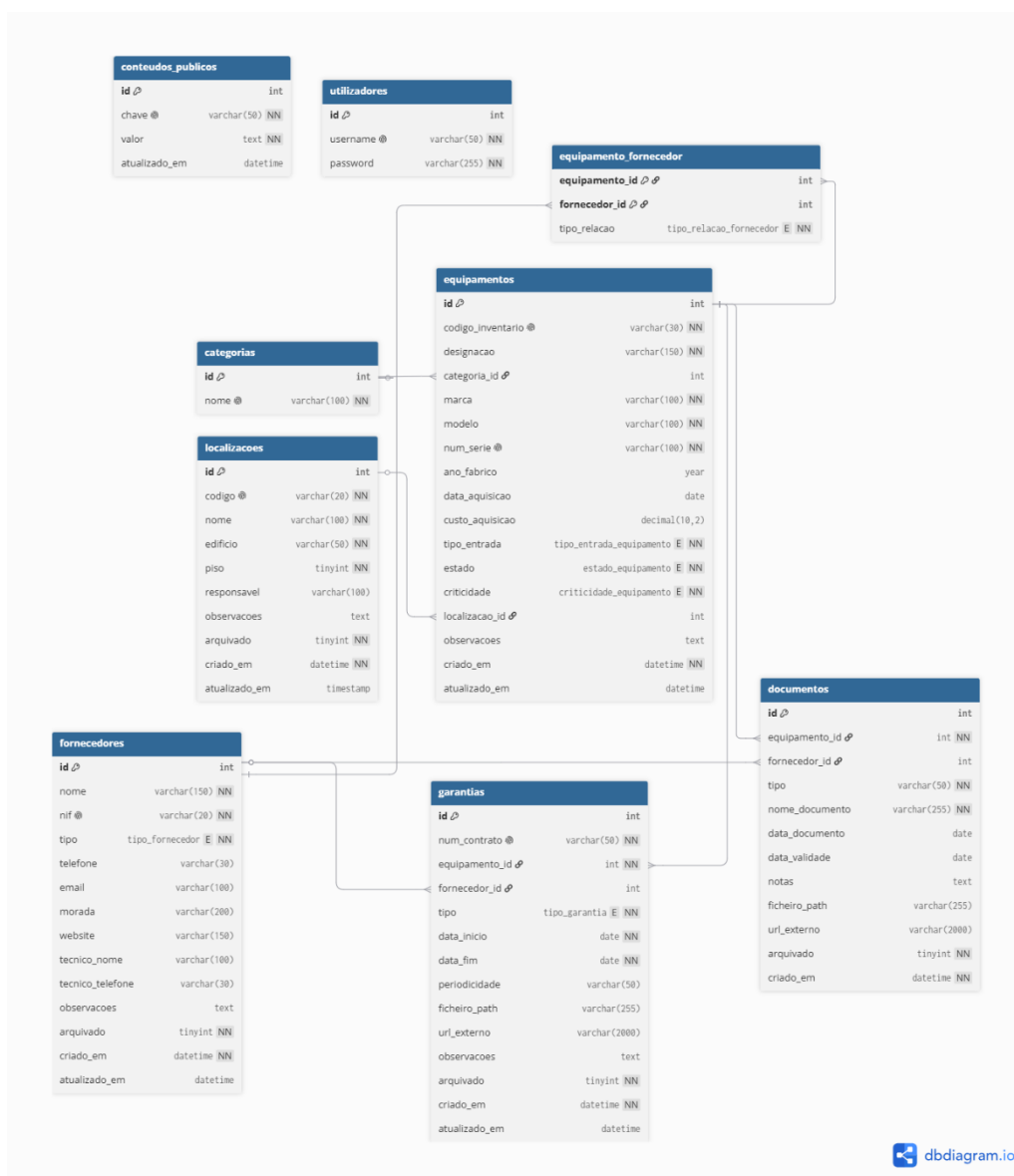


Figura 4 - Gráfico do modelo relacional



De modo a garantir a consistência, fiabilidade e robustez dos dados armazenados, o modelo relacional implementado incorpora um conjunto abrangente de restrições de integridade diretamente ao nível do SQL. Estas restrições desempenham um papel fundamental na prevenção de inconsistências, duplicações indevidas e violações das regras de negócio inerentes ao contexto hospitalar.

No domínio da integridade de entidade, foram definidas múltiplas restrições de unicidade (*UNIQUE*) para assegurar a identificação inequívoca dos registos. Na tabela equipamentos, tanto o campo *codigo_inventario* como o *num_serie* são únicos, garantindo que cada equipamento médico é representado de forma exclusiva no sistema e impedindo a duplicação de dispositivos. De forma semelhante, na tabela fornecedores, o campo *nif* encontra-se sujeito a unicidade, prevenindo a existência de múltiplos registos associados ao mesmo fornecedor. Também a tabela garantias impõe unicidade ao *num_contrato*, assegurando que cada contrato de garantia possui um identificador exclusivo. Na tabela localizacoes, o campo *codigo* é igualmente único, evitando a duplicação de identificadores espaciais hospitalares, como códigos de salas ou unidades clínicas. Por fim, na tabela utilizadores, o campo *username* garante a unicidade das credenciais de autenticação.

Relativamente à integridade de domínio, foram aplicadas restrições que limitam os valores admissíveis em determinados atributos. Na tabela equipamentos, o campo *custo_aquisicao* encontra-se sujeito a uma restrição *CHECK*, impedindo a introdução de valores negativos. Adicionalmente, os campos *estado* e *criticidade* utilizam tipos *ENUM* para restringir os valores possíveis a conjuntos previamente definidos. No caso do estado operacional, apenas são permitidos os valores *Ativo*, *Em manutenção*, *Em calibração*, *Em quarentena* e *Abatido*. Já o nível de criticidade encontra-se limitado às categorias *Baixa*, *Média*, *Alta* e *Suporte de vida*, permitindo classificar os equipamentos segundo a sua relevância clínica. Na tabela garantias, foi ainda implementada uma restrição *CHECK* que assegura que a data de término de uma garantia é obrigatoriamente posterior à respetiva data de início.

No que diz respeito à integridade relacional, o modelo recorre extensivamente a chaves estrangeiras (*Foreign Keys*) com políticas explícitas de propagação. A tabela associativa equipamento_fornecedor utiliza uma chave primária composta por (*equipamento_id*, *fornecedor_id*), garantindo que um mesmo fornecedor não pode ser associado repetidamente ao mesmo equipamento com o mesmo papel relacional. Nas tabelas documentos e garantias, a relação com equipamentos foi configurada com a política *ON DELETE CASCADE*, assegurando que a remoção de um equipamento elimina automaticamente todos os documentos e garantias a ele associados, evitando registos órfãos. Por outro lado, as relações com a tabela fornecedores utilizam a política *ON DELETE SET NULL*, preservando documentos e garantias mesmo após a eliminação do fornecedor, removendo apenas a referência associativa.

Em conjunto, estas restrições reforçam significativamente a integridade semântica e estrutural da base de dados, assegurando que toda a informação armazenada respeita as regras de negócio definidas para o sistema de gestão de inventário clínico.

Restrições de integridade não representáveis no modelo relacional Crow's Foot

O modelo relacional em notação Crow's Foot é uma ferramenta imprescindível para a criação de bases de dados, dado que representa graficamente chaves primárias, chaves estrangeiras, cardinalidade das reações (1:N, N:M) e obrigatoriedade (NOT NULL). Porém,



apresenta algumas limitações no que toca à representação de algumas restrições, dado as mesmas não terem uma representação gráfica direta.

Neste sentido, a primeira restrição que não se encontra representada no modelo descrito encontra-se na tabela de equipamentos na coluna `custo_aquisicao`. A restrição `CK_equipamentos_custo` — `custo_aquisicao >= 0` está representada na imagem abaixo e é do tipo CHECK (validações de valor). A fim de garantir a lógica da gestão do inventário impede o registo de custos de aquisição negativos. Como, o Crow's Foot apenas representa estrutura, isto é, entidades, atributos e relações e não representa regras de valor sobre um único atributo, então não existe símbolo gráfico para "intervalo de valores aceitável".

```
CONSTRAINT `CK_equipamentos_custo` CHECK ((`custo_aquisicao` >= 0))
```

Figura 5 – Restrição de Integridade não Representável no Modelo relacional Crow's Foot 1

Para além disso, na tabela garantias nas colunas `garantias_data_fim` e `garantias_data_inicio`, também está presente uma restrição do tipo CHECK, dado tratar-se da comparação de dois atributos. Está-se a fazer referência à restrição `CK_garantias_datas` — `data_fim > data_inicio`, que garante que nenhuma garantia/contrato é registada com data de fim anterior (ou igual) à data de início. Esta regra relaciona dois atributos da mesma linha contudo, o modelo relacional representa relações entre tabelas, não comparações lógicas entre colunas da mesma tabela.

```
CONSTRAINT `CK_garantias_datas` CHECK ((`data_fim` > `data_inicio`))
```

Figura 6 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 2

Por sua vez, em diversas tabelas é usada uma restrição do tipo restrição de domínio, onde apenas são aceites um conjunto específico de valores (ENUM) com o intuito de representar colunas como estado, criticidade, tipo_entrada, tipo_relacao e tipo. Esta restrição foi implementada a fim de evitar que sejam aceites valores com erros ou redundantes, tome-se como exemplo o estado dos equipamentos (figura 3). Devido ao facto de apenas serem aceites cinco valores definidos, nomeadamente, "Ativo", "Em manutenção", "Em calibração", "Em quarentena" e "Abatido", valores similares como "Activo" ou "Em calibracao" que por vezes constituem erros sintáticos não são aceites, evitando, deste modo, erros na base de dados. Esta restrição não é representável no modelo Crow's Foot pois, o mesmo mostra o tipo de dado da coluna (ex: varchar, int), mas não a lista exaustiva de valores permitidos dentro desse tipo.

```
`estado` enum('Ativo','Em manutenção','Em calibração','Em quarentena','Abatido')  
CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL,
```

Figura 7 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 3

Outra restrição que também não é representável no modelo relacional é ON DELETE CASCADE / ON DELETE SET NULL. A mesma encontra-se em todas as chaves estrangeiras e representa uma ação referencial. Um exemplo da utilização desta restrição é ao apagar um equipamento. Quando se realiza esta ação, os seus documentos e garantias são eliminados automaticamente (CASCADE), mas se um fornecedor for apagado, o equipamento mantém-se e o campo `fornecedor_id` fica NULL em vez de o registo ser destruído. Assim, a notação Crow's Foot representa a cardinalidade da relação (1:N), mas não o comportamento automático do Sistema de Gestão de Bases de Dados quando o registo "pai" é apagado.

```
CONSTRAINT `FK_equipamentos_categoria` FOREIGN KEY (`categoria_id`)  
REFERENCES `categorias` (`id`) ON DELETE SET NULL,
```

Figura 8 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 4



Além disso, o modelo relacional não tem um mecanismo nativo para "expressões regulares" ou validação de formato de string, logo não apresenta validações de formato como email e NIF. Assim, estas validações não foram aplicadas na base de dados, mas na camada aplicacional (PHP) através de FILTER_VALIDATE_EMAIL.

```
function validar_email_fornecedor(string $email): array
{
    $erros = [];
    $email_limpo = trim($email);
    if ($email_limpo !== '') {
        if (!filter_var($email_limpo, FILTER_VALIDATE_EMAIL)) {
            $erros[] = "O E-mail de Assistência Oficial inserido não é válido.";
        } elseif (mb_strlen($email_limpo) > 100) {
            $erros[] = "O E-mail de Assistência Oficial não pode exceder os 100 caracteres.";
        }
    }
    return $erros;
}
```

Figura 9 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 5

Adicionalmente, existem restrições relacionadas com o carregamento de ficheiros que também não podem ser representadas no modelo relacional. No sistema desenvolvido, os documentos técnicos, contratos e garantias submetidos pelos utilizadores devem obrigatoriamente estar no formato PDF e possuir dimensão máxima de 5 MB. Estas validações são efetuadas exclusivamente na camada aplicacional em PHP, através da verificação do tipo MIME e do tamanho do ficheiro recebido no servidor. Tais regras não podem ser expressas em notação Crow's Foot nem em SQL DDL, uma vez que dizem respeito a propriedades físicas do ficheiro transferido e não à estrutura lógica da base de dados.

```
($ficheiro['size'] > 5 * 1024 * 1024)
```

Figura 10 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 6

Outra restrição não representável no modelo relacional diz respeito ao campo ano_fabrico da tabela equipamentos. Este atributo encontra-se limitado a um intervalo dinâmico compreendido entre 1980 e o ano atual acrescido de uma unidade. Esta validação garante a coerência temporal dos dados introduzidos, impedindo o registo de equipamentos com datas de fabrico manifestamente inválidas. Como o limite superior depende da data corrente do sistema, esta regra não pode ser definida de forma estática através de uma restrição CHECK tradicional, sendo implementada na camada aplicacional em PHP.

```
((int) $ano_fabrico < 1980 || (int) $ano_fabrico > $ano_atual + 1)
```

Figura 11 - Restrição de Integridade não Representável no Modelo relacional Crow's Foot 7

Por fim, a última restrição que não é representável no modelo indicado retrata a obrigatoriedade de pelo menos um fornecedor "Fabricante" por equipamento. A mesma encontra-se na tabela equipamento_fornecedor e antes de ser gravada na base de dados é garantida na aplicação (PHP), no momento de inserção/edição do equipamento. Como é uma restrição que depende da existência de pelo menos uma linha numa tabela, não sendo condicionada a um valor específico de outra coluna, então não pode ser representável no modelo Crow's Foot.



Consultas SQL Aplicadas no Trabalho

Nesta subsecção estão representadas duas consultas que estão, efetivamente, implementadas no código PHP da aplicação dentro de instruções PDO::prepare().

Consulta com Junção de tabelas (JOIN)

A seguinte consulta encontra-se localizada em private/views/equipamentos/detalhes.php.

```
$stmt = $ligacao->prepare("
SELECT
    e.*,
    c.nome AS categoria_nome,
    l.nome AS localizacao_nome,
    l.edificio AS localizacao_edificio,
    l.piso AS localizacao_piso
FROM equipamentos e
LEFT JOIN categorias c ON e.categoria_id = c.id
LEFT JOIN localizacoes l ON e.localizacao_id = l.id
WHERE e.id = :id
");
```

Figura 12 - Consulta com junção de tabelas (JOIN)

Esta consulta é executada sempre que um utilizador acede à ficha técnica detalhada de um equipamento. Ao invés de ser necessário realizar três consultas separadas, uma a equipamentos, outra a categorias e outra a localizacoes, a junção (JOIN) permite obter, numa única instrução, todos os dados do equipamento já acompanhados do nome legível da sua categoria e da sua localização (edifício, piso).

Neste sentido a utilização de LEFT JOIN ao invés de INNER JOIN foi propositada. Como categoria_id e localizacao_id podem ser NULL (devido à ação ON DELETE SET NULL definida nas chaves estrangeiras), um equipamento sem categoria ou localização associada continua a ser apresentado normalmente, apenas com esses campos vazios, em vez de desaparecer da consulta.

Adicionalmente, todas as tabelas envolvidas nesta consulta beneficiam de indexação automática das chaves primárias e estrangeiras no MySQL, o que permite otimizar significativamente o desempenho da operação JOIN, mesmo em cenários simulados com cerca de 1500 registos de equipamentos. Esta otimização é particularmente relevante em sistemas hospitalares, onde o tempo de resposta na consulta de dados clínicos deve ser minimizado.

Consulta com Subconsulta (subquery)

A seguinte consulta encontra-se localizada em private/home.php, no indicador "Equipamentos sem documentação associada".



```
$stmtDocs = $ligacao->query("
    SELECT COUNT(*)
    FROM equipamentos e
    WHERE e.id NOT IN (
        SELECT DISTINCT d.equipamento_id FROM documentos d
    )
    AND e.estado != 'Abatido'
");
$semDocumentacao = $stmtDocs->fetchColumn() ?: 0;
```

Figura 13 - Consulta com subconsulta (subquery)

Esta consulta identifica quantos equipamentos do inventário não possuem nenhum documento técnico associado (nem manual, nem certificado de calibração, nem qualquer outro tipo de documentação). A subconsulta interna (SELECT DISTINCT d.equipamento_id FROM documentos d) devolve a lista de todos os IDs de equipamento que têm pelo menos um documento registado.

Para além disso, a consulta externa conta depois os equipamentos cujo id não consta dessa lista (NOT IN), excluindo ainda os equipamentos já abatidos (irrelevantes para este alerta, pois já não estão em circulação). Este valor é apresentado como métrica numérica no dashboard, apoiando a deteção de equipamentos em incumprimento documental. Este mecanismo é relevante, por exemplo, em auditorias de qualidade hospitalar.

Além disso, esta consulta foi propositadamente implementada com NOT IN (subconsulta) em vez do padrão LEFT JOIN ... WHERE x.id IS NULL (anti-join), para demonstrar de forma distinta as duas técnicas de consulta exigidas no guião — junção (secção 3.1) e subconsulta (esta secção) — em vez de duas variações da mesma técnica.

Módulos da Aplicação

A área privada da aplicação encontra-se estruturada em módulos funcionais independentes, cada um diretamente associado a uma entidade ou conjunto de entidades do modelo de dados relacional. Esta organização modular promove uma arquitetura coesa, escalável e de fácil manutenção, permitindo a segregação clara de responsabilidades entre os diferentes componentes do sistema.

O módulo de Equipamentos constitui o núcleo central da aplicação, disponibilizando operações CRUD (*Create, Read, Update, Delete*) completas sobre o inventário clínico. Este módulo permite o registo de novos dispositivos médicos, listagem com filtros por estado operacional, visualização detalhada, edição de registos existentes, bem como processos de abate lógico (*soft delete*) e posterior restauro. Adicionalmente, suporta a associação de fornecedores segundo diferentes papéis funcionais, nomeadamente fabricante, distribuidor e assistência técnica.

O módulo de Fornecedores disponibiliza igualmente operações CRUD completas, permitindo a gestão de entidades externas responsáveis pelo fornecimento, manutenção ou suporte técnico dos equipamentos. Cada fornecedor é registado com identificadores únicos, como o NIF, bem como informação complementar relativa ao tipo de entidade, contactos institucionais e técnico responsável. Este módulo estabelece uma relação muitos-para-muitos com os equipamentos através de uma tabela associativa dedicada.

No módulo de Localizações, é gerida a distribuição espacial dos equipamentos no ambiente hospitalar através de uma organização hierárquica por edifício, piso, serviço clínico e sala.



Cada localização possui um código identificador único, assegurando a rastreabilidade física precisa dos dispositivos médicos dentro da instituição.

O módulo de Documentação permite a gestão centralizada de documentação técnica associada ao inventário, suportando operações CRUD completas sobre ficheiros e referências externas. O sistema aceita o carregamento de documentos em formato PDF, com limite máximo de 5 MB, bem como a associação de hiperligações externas para plataformas de armazenamento como OneDrive ou Google Drive. Cada documento pode ser associado a um equipamento, a um fornecedor, ou a ambos.

O módulo de Garantias e o módulo de Contratos são responsáveis pela monitorização da cobertura legal e contratual dos equipamentos médicos. Estes componentes permitem registar garantias e contratos de manutenção, incluindo datas de início e término, periodicidade contratual e respetivos documentos associados. Adicionalmente, o sistema incorpora mecanismos visuais de alerta para assinalar contratos próximos da expiração ou já expirados.

A aplicação inclui ainda um módulo de Conteúdos Públicos, concebido como *backoffice* editorial para gestão dinâmica de toda a informação disponibilizada na área pública do portal. Este módulo permite modificar textos, descrições institucionais e conteúdos informativos sem necessidade de alterar diretamente ficheiros HTML, recorrendo a uma estrutura flexível baseada em pares chave-valor armazenados na base de dados.

Por fim, a Dashboard atua como painel central de monitorização e análise operacional do sistema. Esta interface disponibiliza indicadores estatísticos atualizados em tempo real, métricas relevantes para a gestão do inventário clínico e visualizações gráficas interativas geradas com recurso à biblioteca Chart.js, com dados extraídos diretamente da base de dados.

Módulo de Equipamentos

O módulo de equipamentos é o núcleo central do sistema. Cada equipamento possui um código de inventário único (ex: EQ-00001-QWA), é classificado por categoria, estado operacional e nível de criticidade clínica (Baixa, Média, Alta, Suporte de vida).

A listagem principal é filtrada por estado através de um grupo de botões na parte superior da página, permitindo ao utilizador alternar rapidamente entre equipamentos ativos, em manutenção, em calibração, em quarentena ou abatidos. O abate é implementado como soft delete (UPDATE estado = 'Abatido') para preservar o histórico relacional.

A associação a fornecedores é feita através da tabela pivot equipamento_fornecedor, que distingue o papel de cada entidade: Fabricante (obrigatório), Distribuidor Comercial (opcional) e Assistência Técnica (opcional). Esta estrutura permite que um equipamento tenha múltiplos fornecedores com papéis diferentes.

O formulário de registo inclui um modal AJAX para criar categorias sem recarregar a página, melhorando a experiência de utilização. Os IDs nos URLs são encriptados com AES-256-CBC para impedir a manipulação direta de parâmetros.

Módulo de Fornecedores

O módulo de fornecedores permite registar e gerir todas as entidades comerciais relacionadas com os equipamentos médicos: fabricantes, distribuidores, empresas de assistência técnica e fornecedores de consumíveis. O NIF é validado como único no sistema (9 dígitos numéricos).



A ficha de detalhe de cada fornecedor apresenta todos os equipamentos associados, facilitando a consulta reversa.

Módulo de Documentação

A gestão documental permite associar ficheiros a equipamentos e/ou fornecedores em duas modalidades: upload real de PDF (até 5 MB, guardado na pasta uploads/ com nome único gerado por `uniqid()`) ou link externo para plataformas cloud (OneDrive, Google Drive). Esta flexibilidade foi desenhada para se adaptar à realidade hospitalar, onde muitos documentos já existem em plataformas colaborativas.

A validação das datas garante que a data de validade nunca é anterior à data de emissão do documento.

Módulo de Garantias e Módulo de Contratos

O módulo de garantias e o módulo de contratos permitem registar contratos de garantia de fábrica e contratos de manutenção preventiva/corretiva. O número de contrato é único no sistema. A restrição `CHECK(data_fim > data_inicio)` é aplicada tanto no SQL como no PHP.

O dashboard apresenta alertas automáticos para garantias expiradas e para garantias a terminar nos próximos 30 dias, permitindo ao gestor agir proativamente antes da interrupção da cobertura.

Dashboard e Gráficos

O dashboard (`home.php`) é a página de entrada da área privada e apresenta uma visão executiva do parque tecnológico hospitalar. Os indicadores são calculados em tempo real através de queries SQL à base de dados e apresentados em dois formatos complementares:

- Cards de métricas numéricas — Total de equipamentos, ativos, em manutenção, em calibração, em quarentena, abatidos, garantias expiradas, garantias a expirar em 30 dias, equipamentos sem documentação e equipamentos de criticidade elevada.
- Gráficos Chart.js — Barras por serviço (JOIN equipamentos/localizacoes), Doughnut por edifício e Barras horizontais de equipamentos de suporte de vida por serviço.

O dashboard inclui ainda um botão para exportar o relatório em formato HTML imprimível (compatível com Ctrl+P / Guardar como PDF) sem dependências de bibliotecas externas como FPDF.

Conteúdos Públicos (Backoffice)

A tabela `conteudos_publicos` implementa um sistema de chave-valor que permite ao administrador editar todos os textos do site público (textos de serviços, contactos, descrições) sem aceder ao código HTML. A página pública (`public/index.php`) lê estes valores dinamicamente através da função `get_conteudos_publicos()` em `funcoes.php`.

Esta separação entre conteúdo e apresentação é uma boa prática de desenvolvimento web que facilita a manutenção e permite que utilizadores não técnicos atualizem o site.



Implementação de Mecanismos de Backend e Segurança

A segurança constitui um dos pilares fundamentais da aplicação desenvolvida, tendo sido particularmente reforçada na camada de backend em PHP. O sistema foi desenhado com o objetivo de garantir a integridade dos dados clínicos, a confidencialidade da informação e o controlo rigoroso de acessos, mitigando vulnerabilidades comuns em aplicações Web.

Autenticação e gestão de sessões

O processo de autenticação baseia-se na utilização das funções nativas `password_hash()` e `password_verify()`, assegurando o armazenamento seguro das palavras-passe através de hashing com `bcrypt`. Desta forma, as credenciais nunca são armazenadas nem transmitidas em texto simples.

Após autenticação bem-sucedida, são criadas sessões através de `session_start()`, sendo armazenadas variáveis de controlo do utilizador autenticado. O acesso à área privada é garantido através da função `redirect_if_not_logged()`, aplicada no início de todas as páginas do backoffice, impedindo o acesso não autorizado. No processo de logout, a sessão é completamente invalidada através de `session_unset()` e `session_destroy()`, garantindo a eliminação de todos os dados temporários do utilizador no servidor.

Validação e depuração de dados

Todos os dados submetidos através de formulários passam por um processo de validação em duas camadas. Na camada cliente, são aplicadas validações em JavaScript com o objetivo de melhorar a experiência do utilizador e fornecer feedback imediato. No entanto, a validação definitiva ocorre no servidor (PHP), sendo considerada a única fonte de verificação fiável.

Nesta etapa são aplicadas regras estritas sobre existência de campos, formatos (através de expressões regulares e `filter_var()`), valores permitidos em ENUM, restrições como NIF com 9 dígitos, datas válidas no formato AAAA-MM-DD, custos não negativos e unicidade de identificadores. Adicionalmente, os dados são normalizados antes da persistência na base de dados, recorrendo a funções como `strtoupper()` para códigos de inventário, `ucwords()` para normalização de nomes e `trim()` para remoção de espaços, bem como funções específicas para conversão de formatos monetários.

Proteção contra SQL Injection

A comunicação com a base de dados MySQL é realizada exclusivamente através da extensão PDO, garantindo o uso obrigatório de *prepared statements*. As queries são definidas previamente com parâmetros nomeados ou posicionais, sendo os valores do utilizador vinculados posteriormente através de `bindParam()` ou `execute()`.

Este mecanismo assegura que os dados introduzidos são interpretados apenas como valores literais pelo SGBD, eliminando a possibilidade de execução de comandos SQL maliciosos e mitigando de forma eficaz ataques de SQL Injection.

Encriptação e proteção de dados sensíveis

Para reforçar a segurança da aplicação, foram implementados mecanismos de cifragem através da extensão OpenSSL em PHP. Em particular, recorreu-se ao algoritmo AES-256-CBC para a encriptação de identificadores transmitidos em URLs e de outros dados sensíveis



do sistema. Este mecanismo impede a enumeração direta de registos e protege a integridade dos dados expostos na camada de apresentação.

Gestão de sessões e controlo de acessos

O acesso à área privada da aplicação encontra-se restrito através de um sistema de controlo de sessões baseado em PHP. Em cada página protegida é validada a existência de uma sessão ativa e os privilégios do utilizador, garantindo que apenas utilizadores autenticados podem aceder às funcionalidades do backoffice.

Caso seja detetada uma tentativa de acesso direto a páginas internas sem autenticação, o sistema bloqueia o pedido e redireciona o utilizador para a página de login. Este mecanismo assegura um controlo centralizado e consistente de permissões ao longo de toda a aplicação.

Upload seguro de ficheiros

Os ficheiros submetidos através da aplicação, nomeadamente documentos em formato PDF, são sujeitos a validação rigorosa quanto ao tipo MIME, extensão e tamanho máximo permitido (5 MB). Para evitar conflitos e substituição indevida de ficheiros, os nomes são gerados automaticamente através da função `uniqid()`, garantindo unicidade no armazenamento e reforçando a segurança do sistema de uploads.



Documentação do Sistema de Scripts (JavaScript)

Para garantir uma interface dinâmica, validações do lado do cliente em tempo real e uma experiência de utilizador (UX) fluida, foi desenvolvido um ecossistema centralizado de scripts em JavaScript nativo (Vanilla JS) e jQuery. O código encontra-se modularizado e protegido contra falhas de execução através de verificações de existência de elementos no DOM (*Document Object Model*).

Abaixo detalham-se as componentes e funções que constituem o sistema:

Módulo 1: Equipamentos

Validação e formatação do código de inventário

Este módulo assegura a validação e normalização do campo de código de inventário (`#codigo_inventario`), impondo um formato institucional rigoroso do tipo EQ-XXXXX-YYY, composto por 5 dígitos numéricos e 3 caracteres alfabéticos.

A função `formatarCodigo(valor)` é responsável pela limpeza e reconstrução da string introduzida pelo utilizador. O processamento inclui remoção de prefixos, conversão para maiúsculas e filtragem de caracteres inválidos através de expressões regulares. A formatação é realizada de forma incremental, garantindo a inserção automática do hífen separador no ponto correto.

Foram também definidos eventos associados (`input`, `focus` e `keydown`) que permitem formatação em tempo real, inserção automática do prefixo obrigatório e proteção contra remoção do identificador estrutural do código.

Validação do ano de fabrico

Este módulo garante a integridade temporal do campo `#ano_fabrico`, impondo restrições baseadas numa lógica de obsolescência técnica.

A validação define um intervalo dinâmico, limitado entre 1980 e o ano atual acrescido de uma unidade, bem como uma restrição adicional de ciclo de vida útil máximo de 40 anos. A validação é reforçada através da API nativa de HTML5 (`setCustomValidity`), permitindo a emissão de mensagens de erro personalizadas em tempo real.

Módulo 3: Comunicação assíncrona (AJAX / Fetch API)

Este módulo permite a criação dinâmica de categorias sem recarregamento da página, recorrendo a comunicação assíncrona com o backend.

O evento `submit` do formulário é interceptado com `preventDefault()`, evitando submissões tradicionais. Para melhorar a experiência de utilização e prevenir requisições duplicadas, o botão de submissão é temporariamente desativado durante o processamento.

Os dados são enviados através de `FormData` para um endpoint PHP dedicado, sendo a resposta tratada como texto e posteriormente convertida para JSON, prevenindo falhas causadas por eventuais warnings do servidor.

Em caso de sucesso, é criado dinamicamente um elemento `Option`, inserido no respetivo `<select>` e automaticamente selecionado, sendo a janela modal fechada através de Bootstrap.



Módulo 4: Integração de bibliotecas jQuery (DataTables e notificações)

Este módulo assegura a compatibilidade com bibliotecas baseadas em jQuery, iniciando-se apenas após verificação da existência do objeto global \$.

As notificações do sistema são geridas através de elementos `.alert-success`, que são automaticamente removidos após um intervalo de 2 segundos com recurso a `setTimeout` e efeitos de `fadeOut`.

A biblioteca DataTables é inicializada sobre tabelas específicas, fornecendo funcionalidades de pesquisa, ordenação e paginação. A configuração inclui ainda localização completa para português, garantindo maior acessibilidade e usabilidade.

Módulo 5: Automação e geração de dados de teste

Este módulo tem como objetivo apoiar o ambiente de desenvolvimento e validação do sistema.

É gerado dinamicamente um botão de preenchimento automático de dados de teste, permitindo a simulação rápida de formulários completos. São utilizados helpers internos para manipulação simplificada do DOM e funções baseadas em `Math.random()` para gerar dados fictícios coerentes, como NIF, códigos de inventário e números de série.

Este módulo integra ainda a biblioteca Flatpickr para simulação de datas, garantindo consistência com os restantes componentes do sistema.

Módulo 6: Autenticação dinâmica (modo desenvolvimento)

Este módulo permite a autenticação rápida em ambiente de desenvolvimento, através do preenchimento automático de credenciais predefinidas. Os eventos de clique associados a botões específicos preenchem diretamente os campos do formulário de login, facilitando testes e validação de perfis de utilizador.

Módulo 7: Componentes globais e geração de gráficos

Este módulo inclui funções globais reutilizáveis, independentes do ciclo `DOMContentLoaded`, permitindo integração com outras partes da aplicação.

A função `removerFicheiroAtual()` permite sinalizar a eliminação de ficheiros associados a registos, ocultando a interface correspondente e limpando campos de upload.

A função `criarGrafico()` atua como uma abstração para a geração de gráficos dinâmicos com a biblioteca `Chart.js`, permitindo parametrização do tipo de gráfico, dados e esquema de cores.

A inicialização dos gráficos é protegida por blocos `try...catch`, garantindo que a ausência de dados injetados pelo backend não compromete a execução global do sistema.



Dificuldades Encontradas e Soluções Adotadas

Encriptação AES e Passagem de IDs nos URLs

A principal dificuldade técnica foi implementar a encriptação dos IDs nos parâmetros GET sem quebrar os redirects e os links. O valor encriptado em Base64/hex contém caracteres como +, /, = que, ao serem incluídos num URL, são interpretados como espaços ou separadores de parâmetros.

A solução foi codificar o valor encriptado em representação hexadecimal (bin2hex / hex2bin) em vez de Base64, eliminando os caracteres problemáticos sem necessidade de urlencode/urldecode adicional.

AJAX para Criação de Categorias

A integração do modal AJAX para criar categorias sem recarregar o formulário de equipamentos revelou-se complexa por causa de conflitos de event listeners globais com o Bootstrap. A solução foi criar um listener exclusivo pelo id do formulário do modal (#formNovaCategoriaExclusivo), evitando interferências com o formulário principal da página.

Gestão de Sessões em Ficheiros Incluídos

Um erro recorrente durante o desenvolvimento foi a chamada duplicada a session_start() quando os ficheiros de include também a invocavam. A solução foi centralizar a lógica de sessão na função start_session() em funcoes.php, que verifica session_status() antes de iniciar uma nova sessão.

Responsividade da Sidebar em Mobile

A sidebar lateral com largura fixa causava overflow em ecrãs pequenos. A solução passou por usar as classes col-md-3/col-lg-2 do Bootstrap com a sidebar e col-md-9/col-lg-10 para o conteúdo principal, e aplicar d-none d-md-block na sidebar em mobile para a ocultar em ecrãs estreitos.

Exportação para PDF sem Bibliotecas Externas

A intenção inicial era usar a biblioteca FPDF para gerar relatórios PDF. No entanto, a FPDF não estava disponível no servidor da ISEP. A solução adotada foi criar uma página HTML otimizada para impressão (exportar_dashboard_pdf.php), com estilos @media print que ocultam botões e menus. O utilizador pode guardar como PDF diretamente através do browser (Ctrl+P → Guardar como PDF), sem depender de bibliotecas externas.



Conclusão e Reflexão Crítica

O desenvolvimento da aplicação web LusoHealth permitiu responder com eficácia ao desafio proposto, resultando num Sistema de Informação e Apoio ao Inventário Hospitalar robusto, dinâmico e centralizado. A transição de um modelo de gestão obsoleto — assente em folhas de cálculo dispersas e registos manuais sem integridade — para uma plataforma digital e relacional traduz-se num ganho imediato de eficiência, rastreabilidade e segurança para a engenharia clínica da instituição de saúde.

Do ponto de vista da modelação de dados, a aplicação rigorosa das Formas Normais (até à 3FN) e o mapeamento detalhado das restrições de integridade revelaram-se fundamentais para erradicar redundâncias e prevenir a corrupção de dados no ecossistema do MySQL. Ficou também demonstrado que, embora o modelo relacional clássico apresente limitações gráficas na notação *Crow's Foot*, a camada aplicacional em PHP funcionou como um complemento indispensável para garantir regras de negócio complexas, tais como a obrigatoriedade de formatos e validações de fluxo organizacional.

No domínio da arquitetura e implementação de software, a divisão estrita entre as áreas pública e privada assegurou o cumprimento das boas práticas de segurança na Web. A adoção da extensão PDO com *Prepared Statements* mitigou por completo vulnerabilidades críticas como o *SQL Injection*, enquanto o controlo de sessões nativo do PHP blindou o acesso à diretoria protegida, garantindo que apenas utilizadores credenciados manipulam o inventário clínico. Adicionalmente, a utilização de bibliotecas de frontend modernas (como o *Bootstrap*, *DataTables* e *Flatpickr*) permitiu construir uma interface com elevado sentido estético, responsiva e otimizada para o quotidiano hospitalar.

Em suma, a concretização do projeto LusoHealth proporcionou uma consolidação prática de competências fundamentais em engenharia de software e bases de dados aplicadas à saúde. O controlo de versões realizado via GitHub foi fulcral para documentar o progresso contínuo e salvaguardar a evolução do código fonte. O sistema final demonstra estar plenamente apto para mitigar os riscos associados à perda de garantias, falhas de documentação ou lapsos de manutenção, posicionando-se como uma ferramenta de valor acrescentado para a gestão moderna de parques tecnológicos em Engenharia Biomédica.

Como perspetiva de evolução futura, o sistema poderá ser expandido através da integração com standards de interoperabilidade clínica, como HL7 e FHIR, permitindo comunicação direta com sistemas hospitalares externos. Adicionalmente, poderão ser implementados mecanismos de auditoria avançada, reforçando a rastreabilidade e conformidade com normas de segurança em ambientes hospitalares reais.



Anexos

Anexo A – Logótipo Aplicação Web LusoHealth



Figura 14 - Logótipo LusoHealth



Anexo B – Declaração de Uso de Inteligência Artificial

Declaração de Utilização de Ferramentas de Inteligência Artificial

No presente relatório, do projeto LusoHealth, realizado no âmbito da Unidade Curricular de Sistemas de Informação e Base de Dados Aplicados à Saúde (SIBDAS), do curso de Licenciatura em Engenharia Biomédica do Instituto Superior de Engenharia do Porto, no ano letivo 2025/2026, utilizaram-se ferramentas de Inteligência Artificial como apoio complementar ao trabalho realizado.

As ferramentas de Inteligência Artificial foram empregues de forma responsável, não substituindo o trabalho académico próprio, mas servindo como suporte em tarefas específicas de natureza auxiliar, nomeadamente:

- **Revisão e correção ortográfica e gramatical:** foram utilizadas ferramentas como ChatGPT e Gemini para identificação e correção de erros ortográficos e melhoria da clareza textual em diferentes secções do relatório;
- **Geração de imagens:** o ChatGPT foi utilizado para a criação de imagens de apoio ao projeto, com o objetivo de ilustrar conceitos e melhorar a apresentação visual do trabalho;
- **Apoio à programação:** foi utilizada a ferramenta Claude para a geração de excertos de código, nomeadamente para implementação de funcionalidades como inserção de dados em listas na base de dados, geração de gráficos e a estruturação de rotinas assíncronas via AJAX para a criação rápida de categorias em segundo plano, sem necessidade de recarregamento de página;
- **Apoio na depuração e otimização de código:** as ferramentas de Inteligência Artificial foram também utilizadas para identificação e correção de erros em partes de código, contribuindo para a melhoria da sua eficiência e funcionamento.

Todas as sugestões fornecidas pelas ferramentas de Inteligência Artificial foram devidamente analisadas, validadas e adaptadas pela autora do projeto, garantindo a sua adequação ao contexto académico e científico do trabalho.

Declara-se ainda que, a utilização destas ferramentas não compromete a autoria nem a integridade científica do projeto, sendo o conteúdo final da inteira responsabilidade da autora.

Porto, 24 de junho de 2026

Autora: Inês Moreira - 1241841